Spécifications Fonctionnelles — Weekook V2

Projet : Weekook — Plateforme de cuisine à domicile **Date** : 2026-10-28 **Version** : 1.2 **Usage** : Référence pour tests fonctionnels, recette et automatisation QA

Table des matières

1. Rôles utilisateurs
2. Authentification
3. Page d'accueil
4. Recherche de kookers
5. Profil public d'un kooker
6. Réservation
7. Dashboard utilisateur
8. Devenir kooker
9. Création d'une offre
10. Modification d'une offre
11. Dashboard kooker
12. Messagerie
13. Favoris
14. Avis et modération
15. Panel d'administration
16. Règles métier transverses
17. Scénarios de test

---

1. Rôles utilisateurs

1.1 Définition des rôles

Rôle	Valeur DB	Description
Utilisateur	`user`	Compte standard, peut réserver
Kooker	`kooker`	Prestataire culinaire, peut créer des offres
Administrateur	`admin`	Gestion complète de la plateforme
Suspendu	`suspended`	Compte désactivé, accès bloqué

1.2 Permissions par rôle

Action	Anonyme	User	Kooker	Admin
Voir l'accueil, recherche, profils	✓	✓	✓	✓
Créer un compte / Se connecter	✓	—	—	—
Effectuer une réservation	×	✓	✓	✓
Gérer ses favoris	×	✓	✓	✓
Laisser un avis	×	✓	✓	✓

Accéder au dashboard utilisateur	×	✓	✓	✓
Devenir kooker	×	✓	×	×
Créer / modifier ses offres	×	×	✓	✓
Accéder au dashboard kooker	×	×	✓	✓
Accepter / refuser des réservations	×	×	✓	✓
Accéder au panel admin	×	×	×	✓

---

2. Authentification

2.1 Inscription

Champs requis :

- Prénom (non vide)
- Nom (non vide)
- Email (format valide, unique dans le système)
- Mot de passe (minimum 8 caractères)
- Confirmation du mot de passe (doit être identique)
- Acceptation des CGU (case à cocher obligatoire)

Règles métier :

- Si l'email est déjà utilisé → erreur `409` : *"Cette adresse email est déjà associée à un compte."*
- Les deux mots de passe doivent être identiques (vérification en temps réel côté frontend)
- Le mot de passe est hashé avec bcrypt (12 rounds) avant stockage
- À la création, le rôle est `user`, `kookerProfileId` est `null`, `isAdmin` est `false`

Indicateur de force du mot de passe (frontend) :

- < 2 caractères → rouge
- 2–7 caractères → orange
- 8–11 caractères → vert
- 12+ caractères → vert plein

Après inscription :

- Le compte est créé avec `emailVerified = false`
- Un email de vérification est envoyé à l'adresse saisie (lien valable 24 heures)
- L'utilisateur est redirigé vers une page d'attente lui demandant de consulter ses emails
- **Pas de connexion automatique** tant que l'email n'est pas vérifié
- Un bouton "Renvoyer l'email de vérification" est disponible en cas de non-réception
- Cliquer sur le lien → `GET /auth/verify-email?token=...` → JWT émis, `emailVerified` → `true`, redirection selon rôle

---

2.2 Connexion

Champs requis : Email, Mot de passe

Règles métier :

- Si email inconnu ou mot de passe incorrect → erreur `401` : *"Email ou mot de passe incorrect."* (message générique pour éviter l'énumération)
- Si `emailVerified = false` → erreur `403` avec code `email_not_verified` : l'utilisateur est redirigé vers l'écran de vérification email avec un bouton "Renvoyer l'email de vérification" (`POST /auth/resend-verification` , limité à 5 req / 15 min)
- JWT stocké dans cookie httpOnly, sécurisé, SameSite strict
- Durée de session : 2 heures (inactivité)

Redirection après connexion :

1. Si paramètre `redirect` présent dans l'URL → aller à cette URL
2. Si `user.role === 'admin'` → `/admin`
3. Si `user.kookerProfileId` existe → `/tableau-de-bord`
4. Sinon → `/`

2.3 Déconnexion

- Supprime le cookie JWT
- Redirige vers `/`

2.3b Modification d'email

Flux :

1. L'utilisateur saisit un nouvel email dans ses paramètres de profil
2. L'ancien email reste actif et utilisable jusqu'à confirmation
3. Le nouvel email est stocké temporairement dans le champ `pendingEmail` de l'utilisateur
4. Un email de confirmation est envoyé à la **nouvelle** adresse (lien valable 24 heures)
5. Cliquer sur le lien → `GET /users/confirm-email-change?token=...` → le champ `email` est mis à jour avec `pendingEmail` , `pendingEmail` est effacé, un nouveau JWT est émis
6. Si le lien expire, l'utilisateur peut relancer la demande depuis ses paramètres
7. Tant que le changement n'est pas confirmé, aucune mise à jour du JWT n'est effectuée

---

2.4 Mot de passe oublié

Flux :

1. Utilisateur saisit son email
2. Si l'email existe : un token de reset est généré (crypto.randomBytes 32), valide 1 heure, sauvegardé en DB (anciens tokens supprimés avant)
3. Un email avec lien de reset est envoyé
4. **Toujours** afficher : *"Si cette adresse est associée à un compte, vous recevrez un lien..."* (empêche l'énumération)
5. Utilisateur clique le lien → saisit nouveau mot de passe
6. Si le token est invalide ou expiré → erreur : *"Ce lien est invalide ou a expiré."*
7. Après reset réussi : token marqué comme utilisé, redirection vers connexion

Rate limiting : 5 requêtes / 15 minutes sur cette route

---

3. Page d'accueil

3.1 Barre de recherche hero

- Zone de saisie avec animation typewriter (suggestions : "Couscous", "cuisine vietnamienne", "Marseille Paella géante")
- Soumission → redirection vers `/recherche?q={texte encodé}`

3.2 Prestations vedettes

- Affiche 4 prestations avec `featured=true` (via `GET /services/search?featured=true&limit=4`)
- Données affichées par carte (`ServiceCard`) : image du service, badge type (COURS/KOOK), titre, avatar + nom + ville du kooker, note, prix, durée
- Skeleton loader pendant le chargement
- Clic sur une carte → `/prestation/:id`

3.3 Carrousel de témoignages

- Témoignages chargés via `GET /testimonials`
- Navigation précédent/suivant (boutons désactivés aux extrémités)
- Tout utilisateur connecté peut soumettre un témoignage (modal) :
  - Champs : Nom (pré-rempli), Rôle/Profil (optionnel), Contenu (min 10 caractères), Note (1–5 étoiles)
  - Après envoi : *"Merci ! Votre témoignage sera visible après validation par notre équipe."*
  - Envoi unique par session (flag `testimonialSent`)

3.4 CTA "Devenir Kooker"

- **Affiché uniquement si** l'utilisateur connecté n'a pas encore de profil kooker (`user.kookerProfileId === null`)
- Pour utilisateur non connecté : boutons "S'inscrire gratuitement" et "Se connecter"
- Pour utilisateur connecté sans profil kooker : bouton "Devenir Kooker" → `/devenir-kooker`

3.5 FAQ

- 5 questions/réponses statiques en accordéon (une seule ouverte à la fois)
- Sujets : fonctionnement, offres disponibles, devenir kooker, qualité/sécurité, tarifs/paiements

---

4. Recherche de prestations

Endpoint : `GET /api/v1/services/search`

Un kooker avec 3 offres = 3 cartes distinctes dans les résultats. L'utilisateur cherche une prestation, pas un kooker.

4.1 Filtres disponibles

Filtre	Type	Comportement
Recherche texte	Saisie libre	Recherche sur titre de la prestation, description, nom kooker, spécialités du kooker (recherche partielle, insensible à la casse : « orient » trouve « Orientale »)
Type	Sélection	COURS / KOOK / Tous — une prestation ressort si sa liste de types contient le type choisi
Difficulté	Sélection	Apparaît uniquement si Type = COURS ; valeurs : Débutant, Intermédiaire, Avancé

Spécialité	Sélection	Liste issue de la config admin. Une prestation ressort si la spécialité figure dans les spécialités **de son kooker ou de la prestation elle-même**
Adresse de la prestation	Adresse (autocomplétion)	« Chez moi » par défaut pour un client connecté, ou autre adresse (voir §4.1b)
Prix min/max	Numérique	En euros, basé sur le prix de la prestation

4.1b Recherche par adresse et zone d'intervention

Principe : la prestation a toujours lieu à l'adresse du client (son domicile ou une autre adresse qu'il indique) — **rien ne se fait chez le kooker**. Chaque kooker définit son adresse, sa ville et une **zone d'intervention** (rayon en km autour de son adresse, voir §8.2 et §11.5). Une prestation ressort si **l'adresse de la prestation est dans la zone d'intervention du kooker**.

Adresse de la prestation (côté client) :

- **Client connecté avec une adresse dans « Mon profil »** : la recherche se fait **par défaut chez lui** (« 🏠 Chez moi »). Bouton **« Autre adresse »** pour en saisir une autre ; bouton **« 🏠 Chez moi »** pour revenir à son domicile
- **Client connecté sans adresse** : message *"Renseignez votre adresse dans « Mon profil » pour chercher chez vous par défaut."* ; il peut saisir une adresse
- **Visiteur** : pas de filtre d'adresse tant qu'il n'en saisit pas une
- Saisie : adresse, ville ou code postal, avec suggestions d'adresses (Base Adresse Nationale)
- Adresse introuvable → aucun résultat et message *"Nous n'avons pas trouvé cette adresse. Vérifiez-la ou choisissez une des suggestions."*

Règles :

- Personne ne saisit de coordonnées : le **système localise lui-même** les adresses (client et kooker). Les coordonnées sont une donnée interne, **jamais affichées ni renvoyées** par l'API
- Aucune distance n'est affichée dans les résultats
- Le client ne choisit pas de distance : seule compte la zone d'intervention du kooker

Exemple : kooker à Lyon, zone 30 km → un client dont l'adresse est à 17 km de Lyon le trouve ; un client à 40 km ne le trouve pas.

Les kookers dont l'adresse et la ville sont introuvables n'apparaissent pas dans une recherche avec adresse. Les profils existants sont localisés automatiquement au démarrage du serveur.

URL : `chezmoi=1` (domicile — l'adresse elle-même n'apparaît pas dans l'URL) ou `adresse=<texte>` ; `lieu=partout` quand un client ayant un domicile cherche sans adresse (le domicile n'est alors pas réappliqué au rechargement).

Logique des filtres :

- Les filtres "en cours de saisie" ne s'appliquent qu'après clic sur "Appliquer"
- Bouton "Réinitialiser" vide tous les filtres et relance la recherche
- Les filtres actifs affichent des tags avec bouton de suppression individuelle
- L'URL est mise à jour à chaque changement (source de vérité)

4.2 Résultats

- Limite par défaut : 12 prestations par page

- Affichage en grille responsive (1 → 2 → 3 → 4 colonnes) — composant `ServiceCard`
- Tri : prestations de kookers vedettes en premier, puis par note décroissante
- Compteur : *"N prestation(s) trouvée(s)"*
- État vide : message *"Aucun résultat"* avec suggestion de modifier les filtres
- Skeleton loader pendant le chargement (8 cartes fantômes)

4.3 Clic sur une carte

→ Navigation vers `/prestation/:id` (ServiceDetailPage)

4b. Détail d'une prestation (`/prestation/:id`)

Endpoint : `GET /api/v1/services/:id`

Sections affichées

1. En-tête : titre, badge type (COURS/KOOK), badges allergènes/régime
2. Galerie photos avec visionneuse (Dialog)
3. Description + détails (durée, max convives, difficulté si COURS)
4. Prix : forfait de base + convives supplémentaires (si applicable)
5. Ingrédients (si `ingredientsList` renseigné)
6. Équipements kooker / contraintes client
7. Encart Kooker : avatar, nom, ville, note, nb avis, spécialités, lien "Voir son profil" → `/kooker/:id`
8. CTA principal : **"Réserver cette prestation"** → `/reservation?service={id}&kooker={kookerProfileId}`

Règles

- Route publique (pas d'authentification requise pour consulter)
- Le bouton "Réserver" redirige vers `/connexion` si l'utilisateur n'est pas connecté (géré par `BookingPage`)

---

5. Profil public d'un kooker

5.1 Données affichées

- Avatar, prénom, nom, ville, note moyenne, nombre d'avis
- Spécialités (badges)
- Biographie
- Types de prestations proposées (COURS / KOOK)

5.2 Offres (services)

- Liste en accordéon, une offre peut être développée à la fois
- Par offre : titre, type, prix, durée, nombre max de convives, description
- Galerie photos avec visionneuse (Dialog plein écran)
- Ingrédients (si renseignés) avec quantités
- **Option d'achat des ingrédients :**
  - Affiché **uniquement si** `ingredientsPricePerGuest > 0`
  - Choix : Client achète / Kooker apporte (+X€/convive)
  - Par défaut : Client achète
- Allergènes et tags régime (végétarien, végan, sans porc)
- Contraintes matérielles (équipements requis du client)

5.3 Planning

- Affichage des disponibilités futures
- Vue calendrier compact avec tooltips

5.4 Avis

- **Seuls les avis avec statut `approved` sont visibles**
- Affichage : nom de l'auteur, note (étoiles), date, commentaire
- Note moyenne et nombre total d'avis affichés en en-tête

5.5 Actions visiteur

- Bouton "Contacter" → ouvre la messagerie (requiert connexion)
- Bouton favori (cœur) → ajoute/retire des favoris (requiert connexion)
- Bouton "Réserver" sur chaque offre → `/reservation/:serviceId`

---

6. Réservation

6.1 Flux en 5 étapes

```
Étape 1 : Sélection date & heure
Étape 2 : Nombre de convives
Étape 3 : Options (ingrédients)
Étape 4 : Récapitulatif & paiement
Étape 5 : Confirmation
```

6.2 Étape 1 — Date & heure

- Seules les dates futures avec des créneaux disponibles sont sélectionnables
- Le calendrier est construit à partir des disponibilités renseignées par le kooker
- Sélectionner une date affiche les créneaux horaires disponibles ce jour-là

6.3 Étape 2 — Nombre de convives

- Entre 1 et `maxGuests` (inclusif)
- Le dépassement de `maxGuests` est bloqué → erreur : *"Le nombre maximum d'invités pour ce service est X"*
- Affichage du prix en temps réel selon la sélection

6.4 Étape 3 — Options ingrédients

- **Affichée uniquement si** `service.ingredientsPricePerGuestInCents > 0`
- Choix :
  - **Client achète** (défaut) — pas de surcoût
  - **Kooker apporte** — surcoût = `ingredientsPricePerGuestInCents × nombre de convives`

6.5 Calcul du prix

COURS :

```
Prix total = priceInCents
           + max(0, guests − 6) × extraGuestPriceInCents
```

```
           + (ingredientsSource === 'kooker' ? ingredientsPricePerGuestInCents ×
guests : 0)
```

Le forfait de base couvre toujours **6 convives** (paramètre configurable admin `kookBaseGuests`, défaut = 6).

KOOK :

```
Prix total = priceInCents
           + max(0, guests − minGuests) × extraGuestPriceInCents
           + (ingredientsSource === 'kooker' ? ingredientsPricePerGuestInCents ×
guests : 0)
```

Le forfait de base couvre `minGuests` convives (défini par le kooker à la création de l'offre).

6.6 Étape 4 — Paiement Stripe

Prérequis :

- Le kooker doit avoir un compte Stripe connecté (`stripeAccountId` renseigné)
- L'onboarding Stripe doit être complet (`stripeOnboardingComplete = true`)
- Si Stripe non configuré côté serveur → erreur `503`
- Si kooker non configuré → erreur `400` : *"Ce kooker n'accepte pas encore les paiements en ligne."*

Flux Stripe :

1. Création d'un `PaymentIntent` avec capture manuelle
2. Le montant est autorisé (pré-réservé) sur la carte du client
3. Le paiement n'est débité qu'à la confirmation par le kooker

6.7 Étape 5 — Confirmation de la demande

Écran de succès : *"Demande envoyée !"* (et non "Réservation confirmée !")

- Le statut initial de la réservation est `pending` — la prestation n'est pas encore confirmée
- Le kooker doit accepter la demande pour que la réservation soit effective
- L'utilisateur reçoit un email de notification dès que le kooker accepte ou refuse

Cycle de vie des statuts :

```
pending             → Demande envoyée, en attente de confirmation kooker
confirmed           → Kooker a accepté, paiement capturé
awaiting_confirmation → Prestation effectuée, en attente de confirmation par le client
completed           → Client a confirmé la réalisation (fonds transférés au kooker)
cancelled           → Annulation (par client ou kooker)
```

Annulation :

- Si `paymentStatus = authorized` → annulation du PaymentIntent (aucun débit)
- Si `paymentStatus = captured` → remboursement Stripe émis

Règle de date passée :

- Impossible de créer une réservation avec une date passée → erreur `400` : *"La date de réservation est dépassée"*

Règle d'unicité :

- Le serveur vérifie l'unicité de la combinaison `(userId, serviceId, date, startTime)` en excluant les réservations annulées
- Si une réservation identique existe déjà → erreur `409` : *"Vous avez déjà une réservation pour ce service à cette date et heure"*

---

7. Dashboard utilisateur

7.1 Carte profil

- Avatar (modifiable par upload — compress → POST `/upload` → PUT `/users/avatar`)
- Prénom, nom, email
- **Si kooker** : bouton "Mon tableau de bord Kooker" → `/kooker-dashboard`
- **Si non kooker** : bouton "Devenir Kooker" → `/devenir-kooker`
- Bouton "Se déconnecter"

7.2 Onglet 1 — Réservations à venir

- Filtre : `status IN (pending, confirmed, awaiting_confirmation)` ET `date >= aujourd'hui`
- Triées par date croissante
- Par carte : avatar kooker, nom, nom du service, badge type (COURS/KOOK), date, heure, nombre de convives, prix total, badge statut
- **Actions disponibles selon statut :**
  - Tous (sauf annulé/terminé) : "Voir les détails", "Annuler"
  - `awaiting_confirmation` : bouton "Confirmer la réalisation" → `PUT /bookings/:id/status` (status=completed)

7.3 Onglet 2 — Historique

- Réservations passées (terminées, annulées, ou date dépassée)
- Triées par date décroissante
- Si `status = completed` et aucun avis laissé pour ce booking → bouton "Laisser un avis"
- Si avis déjà laissé → badge "✓ Avis laissé"

Modal d'avis :

- Note obligatoire (1–5 étoiles)
- Commentaire optionnel
- Soumission → `POST /reviews` (kookerProfileId, bookingId, rating, comment)
- L'avis est créé avec statut `pending` (modération admin requise)
- Toast : *"Avis publié — merci !"*

7.4 Onglet 3 — Favoris

- Chargé via `GET /favorites`
- Grille de kookers favoris
- Bouton cœur pour retirer (`DELETE /favorites/:kookerId`)
- Bouton "Voir le profil" → `/kooker/:id`
- État vide : message avec CTA vers la recherche

7.5 Onglet 4 — Mes infos

- Formulaire : Prénom, Nom, Email, Téléphone
- Sauvegarde : `PUT /users/profile`

- Toast de confirmation : *"Informations mises à jour"*

---

8. Devenir kooker

8.1 Conditions

- L'utilisateur doit être connecté
- L'utilisateur ne doit **pas** déjà être kooker — si oui → erreur `409` : *"Vous etes deja kooker"*

8.2 Formulaire

Champs requis :

- Téléphone
- Adresse
- Ville — autocomplétion de communes
- L'adresse et la ville sont **enregistrées** ; le serveur en déduit la position du kooker (adresse cherchée dans sa commune, sinon centre de la commune). Aucune coordonnée n'est saisie ni affichée
- Biographie/Description
- Spécialités — **sélection parmi la liste admin** (Configuration → Spécialités), pas de saisie libre

Zone d'intervention : 5 / 10 / 20 / 30 / 50 / 100 km autour de son adresse (défaut 30 km), libellé *"Lyon et 30 km autour"*. Le kooker apparaît dans les recherches dont l'adresse de prestation est dans cette zone (§4.1b).

Champs optionnels :

- Années d'expérience (0–50)
- Capacité max (1–100 convives)
- Disponibilité générale (texte libre)
- Professionnel / Entreprise (checkbox — SARL, SAS, SIRET)

Après soumission (`POST /kookers/become`) :

- Profil kooker créé avec `active = false` , `featured = false` , `verified = false`
- Rôle utilisateur mis à jour : `user` → `kooker`
- Contexte auth rechargé (kookerProfileId disponible)
- Toast : *"Votre profil Kooker a été créé ! Il sera visible après validation."*
- Redirection vers `/kooker-dashboard`

---

9. Création d'une offre

9.1 Types d'offres

- **COURS** : cours de cuisine donné au domicile du client ou chez le kooker
- **KOOK** : repas préparé et servi au domicile du client
- Le kooker peut créer les deux types simultanément

9.2 Configuration chargée depuis l'admin

Via `GET /admin/config/public` :

- `commissionKours` (%) et `commissionKook` (%)
- Liste des spécialités (seules valeurs autorisées, voir §16.9)
- `kookBaseGuests` (défaut = 6)
- Liste des unités de mesure

- Texte des tooltips (allergènes, commission)

9.3 Champs communs COURS et KOOK

Champ	Requis	Contrainte
Titre	Oui	Non vide
Description	Oui	Non vide
Prix (€)	Oui	≥ 0, stocké en centimes
Prix convive supplémentaire (€)	Oui	≥ 0
Durée (minutes)	**COURS uniquement**	≥ 1. Pour les KOOK, la durée n'est ni demandée ni affichée (MVP : elle se discute en messagerie) ; enregistrée à 0
Nombre max de convives	Oui	≥ 1
Nombre min de convives (KOOK)	Non	Définit le forfait de base
Spécialités	Non	Sélection parmi la liste admin (§16.9)
Allergènes	Non	Parmi : Gluten, Lactose, Arachide, Fruits à coques, Œuf, Fruits de mer, Autres
Tags régime	Non	Végétarien, Végan, Sans porc
Photos	Non	Fichiers uploadés, compressés, URLs stockées

9.4 Champs spécifiques COURS

- Difficulté : Débutant / Intermédiaire / Avancé
- Lieu du cours (texte libre)

9.5 Ingrédients

- Liste d'ingrédients : nom, quantité, unité (issue de la config)
- Base de calcul : nombre de convives de référence
- **Prix par convive** : optionnel — si > 0, l'option "Kooker apporte les ingrédients" est disponible pour le client lors de la réservation

9.6 Équipements

- Équipement fourni par le kooker (liste)
- Équipement requis du client / contraintes (liste)

9.7 Validation à la soumission

- Au moins un type sélectionné
- Tous les champs requis renseignés pour chaque type sélectionné
- Si COURS et KOOK sélectionnés : les champs KOOK sont désactivés tant que COURS n'est pas complet

Soumission : `POST /services` par type (deux appels si les deux types sélectionnés) **Succès :** Toast *"Service créé avec succès !"* → redirection `/kooker-dashboard?tab=services`

---

10. Modification d'une offre

- Chargement initial : `GET /services/:id`
- Formulaire identique à la création, pré-rempli
- Seul le propriétaire du service peut le modifier → sinon erreur `403` : *"Vous ne pouvez modifier que vos propres services"*
- Soumission : `PUT /services/:id`
- Les photos et items de menu sont remplacés intégralement
- Spécialités : une spécialité déjà enregistrée mais retirée de la liste admin s'affiche « hors liste » (pointillés) et peut seulement être retirée
- **Succès :** Toast *"Service modifié avec succès !"* → redirection `/kooker-dashboard?tab=services`

---

11. Dashboard kooker

11.1 Statistiques (en-tête)

Chargées via `GET /kookers/dashboard/stats` :

- Total réservations reçues
- Réservations en attente
- Chiffre d'affaires total (réservations `completed`)
- Nombre de services actifs
- Note moyenne
- Nombre total d'avis approuvés

11.2 Onglet Réservations

- Chargé via `GET /bookings/kooker`
- Filtre par statut : tous / pending / confirmed / completed / cancelled / awaiting_confirmation
- Par carte : avatar client, nom, service, date/heure, nb convives, prix, statut

Actions selon statut :

- `pending` :
  - **Accepter** → `PUT /bookings/:id/status` (status=confirmed) → paiement capturé, email au client
  - **Refuser** → modal avec raisons prédéfinies (indisponible, nb convives, distance, délai, menu, autre réservation, autre raison) → `PUT /bookings/:id/status` (status=cancelled) + message automatique au client

Stripe Connect :

- Si kooker non connecté à Stripe → bannière d'avertissement + bouton "Configurer les paiements"
- `POST /stripe/connect/onboard` → URL d'onboarding Stripe → redirection
- Retour depuis Stripe avec paramètre `stripe=return` ou `stripe=refresh`

11.3 Onglet Mes offres (services)

- Liste de tous les services du kooker

- Par service :
  - Titre, type, prix, durée (COURS uniquement), max convives, nombre de photos
  - Toggle actif/inactif (`PUT /services/:id`)
  - Bouton "Modifier" → `/edit-menu/:id`
  - Bouton "Supprimer" → confirmation → `DELETE /services/:id`
  - Sélection de la photo de carte (`PUT /services/card-image/:imageId`)

Services inactifs : N'apparaissent pas dans les résultats de recherche

11.4 Onglet Planning

- Gestion des disponibilités en batch
- Chaque disponibilité : date (YYYY-MM-DD), heure début (HH:MM), heure fin (HH:MM)
- `PUT /availability` remplace toutes les disponibilités futures d'un coup
- Seules les disponibilités futures (date ≥ aujourd'hui) sont affichées et gérées

11.5 Onglet Profil kooker

- Chargé via `GET /kookers/profile/me` (profil complet du kooker connecté, adresse incluse, même si le profil n'est pas encore validé)
- Champs modifiables : biographie, spécialités (liste admin, §16.9), ville (autocomplétion), adresse, **zone d'intervention** (5 à 100 km autour de l'adresse) — modifier l'adresse ou la ville recalcule la position du kooker, type(s) de prestation, expérience, téléphone, adresse, statut entreprise
- Avatar modifiable (upload → compress → `POST /upload` → `PUT /users/avatar`)
- Sauvegarde : `PUT /kookers/profile` + `PUT /users/profile`

---

12. Messagerie

12.1 Règles

- Un utilisateur ne peut pas s'envoyer un message à lui-même → erreur `400`
- Le destinataire doit exister → sinon erreur `404`
- Un service doit être lié à la conversation (`serviceId` obligatoire)
- Plusieurs conversations possibles avec le même partenaire si services différents

12.2 Conversations

- Listées via `GET /messages/conversations`
- Regroupées par partenaire + service
- Nombre de messages non lus affiché par conversation
- Messages automatiques générés lors des événements de réservation (acceptation, refus, modification, annulation)

12.3 Lecture

- `GET /messages/conversation/:userId` — marque automatiquement les messages reçus comme lus
- Limite : 500 derniers messages par conversation

12.4 Suppression

- Un utilisateur peut supprimer ses propres messages ou les messages reçus
- Suppression d'une conversation : supprime tous les messages entre les deux parties (filtre optionnel par service)

---

13. Favoris

- Ajout : `POST /favorites/:kookerId` — idempotent (pas d'erreur si déjà favori)
- Suppression : `DELETE /favorites/:kookerId` — erreur `404` si non trouvé
- Contrainte unique : un utilisateur ne peut pas mettre le même kooker en favori deux fois
- Le kooker doit exister → sinon erreur `404`

---

14. Avis et modération

14.1 Soumission d'un avis (client → kooker)

Conditions :

- Utilisateur connecté
- Ne peut pas laisser un avis sur son propre profil → erreur `400`
- Si `bookingId` fourni :
  - La réservation doit appartenir à l'utilisateur
  - Le statut doit être `completed`
  - Un seul avis par réservation → erreur `409` : *"Vous avez déjà laissé un avis pour cette réservation"*
- Note obligatoire (1–5)
- Commentaire optionnel

Après soumission :

- Avis créé avec `status = 'pending'`
- **Non visible publiquement** avant validation admin
- Email de notification envoyé à tous les administrateurs (Resend)
- La note moyenne du kooker **n'est pas mise à jour** immédiatement

14.2 Avis kooker → client

- Uniquement après une réservation `completed`
- Le kooker peut noter son client **indépendamment** — aucun prérequis sur l'avis du client
- Un seul avis par réservation → erreur `409`
- Statut initial : `pending` (modération admin requise, comme les avis client)
- Endpoint : `POST /reviews/kooker-to-user` avec `{ bookingId, rating, comment }`

14.3 Modération admin

La page `/admin/avis` affiche les deux types d'avis (`user_to_kooker` et `kooker_to_user`).

Filtres disponibles :

- Par statut : En attente / Approuvé / Rejeté
- Par type : Tous / Client → Kooker / Kooker → Client

Approuver un avis :

- `PUT /admin/reviews/:id/status` avec `{ status: 'approved' }`
- L'avis devient visible
- **Pour `user_to_kooker` uniquement** : la note moyenne du kooker est recalculée
- Les avis `kooker_to_user` n'affectent pas la note du kooker

Rejeter un avis :

- `PUT /admin/reviews/:id/status` avec `{ status: 'rejected' }`
- L'avis est supprimé de la base
- La note du kooker est recalculée uniquement si c'était un avis `user_to_kooker` approuvé

Supprimer un avis :

- `DELETE /admin/reviews/:id`
- La note est recalculée uniquement si c'était un avis `user_to_kooker`

Formule de recalcul (avis user_to_kooker uniquement) :

```
newRating = Math.round((moyenne des ratings des avis user_to_kooker approved) × 10) / 10
reviewCount = nombre d'avis user_to_kooker approved
```

---

15. Panel d'administration

Accès : Utilisateurs avec `isAdmin = true` uniquement. Redirection automatique vers `/admin` après connexion.

15.1 Gestion des utilisateurs

- Recherche par email, prénom, nom
- Filtre par rôle : user / kooker / suspended
- Pagination : 20 par page
- Actions : activer/désactiver le statut admin, supprimer le compte (irréversible)

15.2 Gestion des kookers

- Recherche par nom, email, ville
- Affichage : ID, nom, ville, types, note, nb avis, nb services, nb réservations
- Actions :
  - Toggle **vedette** (★) — apparaît sur la homepage
  - Toggle **vérifié** (✓) — badge affiché sur le profil
  - Toggle **actif** — kooker visible dans la recherche
- Modération des avis via modal (voir §14.3)

15.3 Configuration de la plateforme

Valeurs modifiables (stockées en DB) :

Clé	Type	Description
`commissionKours`	Nombre (%)	Commission prélevée sur les COURS
`commissionKook`	Nombre (%)	Commission prélevée sur les KOOK
`kookBaseGuests`	Nombre	Forfait de base KOOK (nb de convives inclus)
`tooltipFruitsDesMer`	Texte	Texte info-bulle allergène "Fruits de mer"
`tooltipCommission`	Texte	Texte info-bulle commission (simulation revenus)

`specialties`	Liste	Spécialités culinaires — **seules valeurs acceptées** pour les kookers et les offres (§16.9)
`cities`	Liste	Ancienne liste de villes — n'est plus utilisée par la recherche (remplacée par la recherche par lieu, §4.1b)
`allergens`	Liste	Allergènes disponibles
`serviceTypes`	Liste	Types de service
`units`	Liste	Unités de mesure pour les ingrédients

15.3b Messages de contact

- Les messages envoyés via le formulaire `/contact` sont enregistrés en base (table `contact_messages`)
- Rubrique **Admin → Messages contact** (`/admin/messages-contact`) : du plus récent au plus ancien, avec nom, email (lien `mailto:` pour répondre), sujet, message, date
- Messages non lus mis en évidence + compteur ; actions : marquer lu / non lu, supprimer

15.4 KPIs et analytique

- `GET /admin/kpis?period=day|week|month` : revenus, utilisateurs, réservations, kookers + delta vs période précédente
- `GET /admin/business-charts` : acquisition sur 12 semaines, répartition des statuts, top kookers, métriques de santé
- `GET /admin/tech-stats` : compteurs DB, taille des uploads, logs d'erreurs, temps de page

15.5 Pages frontend — liste complète

Route	Composant	Description
`/`	HomePage	Accueil
`/recherche`	SearchPage	Recherche de prestations
`/prestation/:id`	ServiceDetailPage	Détail d'une prestation
`/kooker/:id`	KookerProfilePage	Profil public kooker
`/connexion`	LoginPage	Connexion / Inscription
`/verifier-email`	VerifyEmailPage	Vérification d'email après inscription
`/confirmer-email`	ConfirmEmailChangePage	Confirmation de changement d'email
`/contact`	ContactPage	Formulaire de contact — message enregistré et visible dans l'admin ; confirmation par le bloc *"Message envoyé !"* uniquement (pas de toast)
`/tableau-de-bord`	UserDashboardPage	Dashboard utilisateur
`/kooker-dashboard`	KookerDashboardPage	Dashboard kooker
`/devenir-kooker`	BecomeKookerPage	Formulaire devenir kooker

`/creer-offre`	CreateMenuPage	Créer une offre
`/modifier-offre/:id`	EditMenuPage	Modifier une offre
`/reservation`	BookingPage	Tunnel de réservation
`/reservation/:id`	BookingDetailPage	Détail réservation
`/messagerie`	MessagesPage	Messagerie
`/mon-profil`	UserProfilePage	Profil utilisateur
`/admin`	AdminDashboardPage	Tableau de bord admin
`/admin/utilisateurs`	AdminUsersPage	Gestion utilisateurs
`/admin/kookers`	AdminKookersPage	Gestion kookers
`/admin/avis`	AdminReviewsPage	Modération des avis
`/admin/reservations`	AdminBookingsPage	Suivi réservations
`/admin/services`	AdminServicesPage	Catalogue services
`/admin/temoignages`	AdminTestimonialsPage	Modération témoignages
`/admin/faq`	AdminFaqPage	Gestion FAQ
`/admin/messages-contact`	AdminContactMessagesPage	Messages du formulaire de contact
`/admin/configuration`	AdminConfigPage	Configuration dynamique

---

16. Règles métier transverses

16.1 Prix

- Stockés en **centimes** (integer) en base de données
- Affichés en euros (÷ 100), format : `"25,50 EUR"`
- Arrondi aux centimes (2 décimales)

16.2 Durées

- Stockées en **minutes** (integer) en base de données
- Affichées en format humain : "1h30" ou "45 min"
- Affichées **uniquement pour les COURS** (cartes, fiche prestation, profil kooker, réservation, dashboard, admin) ; jamais pour les KOOK, y compris les anciens KOOK qui ont une durée enregistrée

16.3 Champs JSON en MySQL

- `specialties` , `type` , `allergens` , `constraints` , `equipmentKooker` , `ingredientsList` : stockés en JSON
- Parsés à la lecture (peuvent arriver sous forme de string ou d'objet selon le driver)
- Toujours enregistrés comme **listes JSON natives** (jamais une liste sérialisée en texte) : les filtres de recherche (`array_contains`) ne trouvent que les listes. Au démarrage, le serveur convertit en

listes les spécialités kooker encore stockées en texte

16.4 Emails (Resend — fire-and-forget)

Déclencheur	Destinataire
Inscription	Client (bienvenue)
Nouvelle réservation	Kooker
Réservation confirmée	Client
Réservation annulée	Client ET Kooker
Réservation modifiée	Client ET Kooker
Prestation terminée	Kooker
Nouvel avis en attente	Tous les admins
Réinitialisation mdp	Client
Nouveau message	Destinataire

Les emails sont envoyés de manière asynchrone (non bloquant). Un échec d'envoi n'empêche pas l'action principale.

16.5 Rate limiting

Route	Limite
`POST /auth/login`	50 req / 15 min
`POST /auth/register`	50 req / 15 min
`POST /auth/forgot-password`	5 req / 15 min
`POST /auth/resend-verification`	5 req / 15 min
`POST /contact`	5 req / 15 min

16.8 Codes HTTP d'autorisation

Code	Signification	Cas d'usage
`401`	Non authentifié	Cookie JWT absent, invalide ou expiré
`403`	Interdit (rôle insuffisant)	Utilisateur authentifié mais sans le rôle requis (`requireKooker`, `requireAdmin`)

Les middlewares `requireKooker` et `requireAdmin` retournent **403** (Forbidden) et non 401, car l'utilisateur est bien identifié mais n'a pas les permissions nécessaires.

16.9 Spécialités

- Kookers (devenir kooker, profil) et offres (création, modification) choisissent leurs spécialités **uniquement dans la liste admin** (`specialties`), via des boutons à cocher

- **Vérification serveur** : toute spécialité hors liste est refusée (`400` : *"Spécialité non proposée : X. Choisissez dans la liste."*)
- Exception : une spécialité **déjà enregistrée** sur le profil ou l'offre reste acceptée (affichée « hors liste ») afin de ne pas bloquer la sauvegarde ; elle peut être retirée mais pas ajoutée à nouveau
- Les comparaisons sont exactes (majuscules et accents compris) : « Italien » ≠ « Italienne »

16.10 Navigation (header et footer)

- **Header** : Accueil, FAQ, A propos (la page `/tarification` existe mais n'est plus dans le menu)
- **Footer** :
  - *Découvrir* : Trouver une expérience (`/recherche`), Se connecter / s'inscrire (`/connexion`), Pourquoi devenir Kooker ? (`/avantages`)
  - *Aide & Confiance* : FAQ, Confiance & sécurité (`/confiance`), Charte du Kooker (à venir)
  - *Weekook* : A propos, Nous contacter (`/contact`)
  - Bas de page : © 2026 Weekook · Mentions légales · CGU · Politique de confidentialité · Politique de cookies (pages à venir) ; icônes Instagram et TikTok ; bloc newsletter

16.6 Images

- Upload via `POST /upload`
- Compression avant envoi (côté frontend)
- Stockées dans `/uploads/` sur le serveur
- Image de carte : une seule par kooker (les autres sont désélectionnées automatiquement)

16.7 Pagination

- Par défaut : 12 résultats (recherche kookers), 20 résultats (admin)
- Maximum : 50 par page
- Réponse inclut : `data[], total, page, limit, totalPages`

---

17. Scénarios de test

BLOC A — Authentification

#	Scénario	Précondition	Actions	Résultat attendu
A01	Inscription réussie	Aucun compte	Remplir formulaire valide (prénom, nom, email unique, mdp 8+ car, cgU cochée)	Connexion automatique, redirection /
A02	Email déjà utilisé	Compte existant avec email X	Tenter d'inscrire avec email X	Erreur : *"Cette adresse email est déjà associée à un compte."*
A03	Mots de passe différents	—	Saisir password ≠ confirm	Bouton désactivé ou erreur "Les mots de passe ne correspondent pas"
A04	Mot de passe < 8 car	—	Saisir 7 caractères	Indicateur rouge, soumission bloquée

A05	CGU non cochée	—	Formulaire valide sans cocher CGU	Soumission bloquée
A06	Connexion réussie	Compte existant	Email + mdp corrects	Connexion + redirection selon rôle
A07	Mauvais mot de passe	Compte existant	Email correct + mauvais mdp	Erreur : *"Email ou mot de passe incorrect."*
A08	Email inconnu	—	Email inexistant	Erreur : *"Email ou mot de passe incorrect."* (même message)
A09	Connexion admin	Compte admin	Connexion avec compte admin	Redirection vers `/admin`
A10	Connexion kooker	Compte kooker	Connexion avec compte kooker	Redirection vers `/tableau-de-bord`
A11	Déconnexion	Connecté	Clic "Se déconnecter"	Cookie supprimé, redirection `/`
A12	Reset mdp — email valide	Compte existant	Saisir email connu	Message succès générique, email envoyé
A13	Reset mdp — email inconnu	—	Saisir email inconnu	**Même message succès** (anti-énumération)
A14	Token reset expiré	Token > 1h	Cliquer lien reset	Erreur : *"Ce lien est invalide ou a expiré."*

BLOC B — Recherche

#	Scénario	Actions	Résultat attendu
B01	Recherche par texte	Saisir "marseille" dans la barre de recherche	Kookers de Marseille affichés
B02	Filtre type COURS	Sélectionner COURS	Seules les offres COURS affichées (non vide si des COURS existent)
B02b	Filtre type KOOK	Sélectionner KOOK ou cliquer le bloc KOOK de l'accueil	Seules les offres KOOK affichées
B03	Filtre type COURS + difficulté	COURS + Débutant	Kookers avec COURS niveau Débutant
B04	Filtre difficulté sans COURS	Sélectionner difficulté sans sélectionner COURS	Filtre difficulté non visible / inactif
B05	Filtre prix	Min = 50€, Max = 100€	Kookers dont le prix min d'offre est entre 50 et 100€

B06	Aucun résultat	Filtres très restrictifs	Message "Aucun résultat", suggestion de modifier les filtres
B07	Réinitialiser les filtres	Clic "Réinitialiser"	Tous les filtres vidés, recherche relancée
B08	URL préserve les filtres	Appliquer filtres + copier URL	Recharger l'URL → mêmes filtres appliqués (y compris le lieu)
B09	Filtre spécialité	Choisir une spécialité de la liste	Offres dont le kooker **ou** l'offre a cette spécialité
B10	Texte partiel sur spécialité	Saisir "pâtiss"	Kookers avec la spécialité « Pâtisserie »
B11	Adresse dans la zone du kooker	Kooker Marseille (30 km) ; adresse = 10 cours Maréchal Foch, Aubagne	Les offres du kooker sont affichées
B12	Adresse hors zone	Kooker Marignane (30 km) ; adresse à Aubagne (≈ 36 km)	Offres de ce kooker absentes
B13	Chez moi par défaut	Client connecté avec une adresse dans « Mon profil » : ouvrir `/recherche`	« 🏠 Chez moi » appliqué, résultats filtrés sur son domicile
B14	Chez moi sans adresse	Connecté sans adresse de profil	Message invitant à renseigner l'adresse ; saisie d'une adresse possible
B15	Autre adresse	Client avec domicile → « Autre adresse » → saisir une adresse → Appliquer	Résultats pour cette adresse ; « 🏠 Chez moi » permet de revenir au domicile
B16	Adresse introuvable	Saisir « zzzz introuvable » → Appliquer	Aucun résultat, message *"Nous n'avons pas trouvé cette adresse…"*
B17	Pas de coordonnées ni de distance	Toute recherche avec adresse	Aucune distance affichée ; l'API ne renvoie ni latitude ni longitude

---

BLOC C — Réservation

#	Scénario	Précondition	Actions	Résultat attendu
C01	Réservation COURS standard	Connecté, kooker Stripe ok	Sélectionner date, 2 convives, confirmer paiement	Booking créé (status=pending), email au kooker

C02	Réservation KOOK forfait	Connecté	4 convives pour offre minGuests=6	Prix = priceInCents (forfait, pas de supplément)
C03	Réservation KOOK + convives sup	Connecté	8 convives pour offre minGuests=6	Prix = priceInCents + 2 × extraGuestPriceInCents
C04	COURS avec 6 convives	—	Exactement 6 convives	Prix = priceInCents (forfait exact)
C05	COURS avec 7 convives	—	7 convives	Prix = priceInCents + 1 × extraGuestPriceInCents
C06	Dépassement maxGuests	maxGuests = 8	Saisir 9 convives	Erreur : *"Le nombre maximum d'invités pour ce service est 8"*
C07	Option ingrédients visible	ingredientsPricePerGuest > 0	Étape 3	Choix affiché
C08	Option ingrédients absente	ingredientsPricePerGuest = 0	Étape 3	Aucun choix affiché, étape skippée ou transparente
C09	Ingrédients kooker → surcoût	Choisir "Kooker apporte"	Récapitulatif	Prix = base + ingredientsPrice × guests
C10	Date passée	—	Sélectionner date < aujourd'hui	Bloqué (date non sélectionnable)
C11	Kooker Stripe non configuré	kooker sans Stripe	Tenter réservation	Erreur : *"Ce kooker n'accepte pas encore les paiements en ligne."*
C12	Annulation avant confirmation kooker	status=pending, paymentStatus=authorized	Client annule	PaymentIntent annulé, aucun débit, status=cancelled
C13	Annulation après confirmation	status=confirmed, paymentStatus=captured	Client annule	Remboursement Stripe émis, status=cancelled
C14	Kooker accepte	status=pending	Kooker clique "Accepter"	status=confirmed, paiement capturé, email client
C15	Kooker refuse	status=pending	Kooker clique "Refuser" +	status=cancelled, message automatique

			raison	au client
C16	Confirmation prestation	status=awaiting_confirmation	Client clique "Confirmer la réalisation"	status=completed, fonds transférés au kooker
C17	Réservation non connecté	Anonyme	Cliquer "Réserver"	Redirection vers `/connexion?redirect=...`

BLOC D — Offres (services)

#	Scénario	Précondition	Actions	Résultat attendu
D01	Créer une offre COURS	Kooker connecté	Remplir formulaire COURS valide	Service créé, redirection dashboard
D02	Créer une offre KOOK	Kooker connecté	Remplir formulaire KOOK valide	Service créé, redirection dashboard
D03	Créer COURS + KOOK	Kooker connecté	Sélectionner les deux types	Deux services créés
D04	Champs requis manquants	—	Soumettre formulaire incomplet	Erreur de validation, formulaire non soumis
D05	Prix négatif	—	Saisir prix = -5	Bloqué (min = 0)
D06	Durée = 0 (COURS)	—	Saisir durée = 0 sur un COURS	Bloqué (min = 1 minute)
D06b	Pas de durée KOOK	Kooker connecté	Créer / modifier un KOOK	Aucun champ durée ; durée non affichée ensuite
D13	Spécialités	Kooker connecté	Créer une offre	Seules les spécialités de la liste admin sont proposées
D14	Spécialité hors liste via API	Kooker connecté	`POST /services` avec une spécialité inexistante	Erreur 400 *"Spécialité non proposée..."*
D15	Ancienne spécialité conservée	Offre avec spécialité retirée de la liste	Modifier l'offre sans toucher aux spécialités	Sauvegarde OK ; spécialité affichée « hors liste »
D07	Modifier son offre	Propriétaire	Modifier et soumettre	Service mis à jour, toast confirmation
D08	Modifier l'offre d'un autre	Autre kooker	Appel `PUT /services/:id`	Erreur 403
D09	Désactiver une offre	Kooker connecté	Toggle inactif	Offre disparaît des résultats de recherche

D10	Supprimer une offre	Kooker connecté	Confirmer suppression	Service supprimé, plus dans la liste
D11	Ingrédients avec prix	Créer offre avec ingredientsPrice > 0	Afficher profil	Option "Kooker apporte" visible lors de la réservation
D12	Ingrédients sans prix	ingredientsPrice = 0	Afficher profil	Aucune option ingrédients

---

BLOC E — Avis

#	Scénario	Précondition	Actions	Résultat attendu
E01	Avis après réservation completed	Booking completed, pas encore d'avis	Laisser avis (4 étoiles)	Avis créé (status=pending), email admins
E02	Avis sur réservation non terminée	Booking status=confirmed	Tenter de laisser un avis	Erreur : prestation non confirmée
E03	Double avis même réservation	Avis déjà soumis pour bookingId X	Tenter un second avis	Erreur 409 : *"Vous avez déjà laissé un avis..."*
E04	Auto-avis	User = kooker du profil	Tenter un avis	Erreur : *"Vous ne pouvez pas laisser un avis sur votre propre profil"*
E05	Avis non visible avant approbation	Avis pending	Consulter profil kooker	Avis absent (non affiché)
E06	Avis visible après approbation	Admin approuve	Consulter profil kooker	Avis affiché, note recalculée
E07	Avis rejeté	Admin rejette	Consulter DB	Avis supprimé, note non modifiée
E08	Note recalculée après approbation	2 avis approuvés (3★ et 5★)	Admin approuve 3ème avis (4★)	Note = (3+5+4)/3 = 4,0

BLOC F — Administration

#	Scénario	Précondition	Actions	Résultat attendu
F01	Accès admin sans rôle	User standard	Accéder à `/admin`	Redirection ou erreur 403

F02	Suspension d'un utilisateur	Admin connecté	Changer rôle → suspended	Utilisateur ne peut plus se connecter
F03	Toggle kooker vedette	Admin connecté	Activer "featured" pour un kooker	Kooker apparaît sur la homepage
F04	Toggle kooker vérifié	Admin connecté	Activer "verified"	Badge vérifié sur le profil
F05	Désactiver un kooker	Admin connecté	Toggle actif → inactif	Kooker n'apparaît plus dans la recherche
F06	Modifier commission KOURS	Admin connecté	Changer commissionKours à 25%	Nouveau taux utilisé dans les simulations de revenus
F07	Ajouter une spécialité	Admin connecté	Ajouter "Japonaise" à la liste spécialités	Proposée aux kookers (profil, offres) et dans le filtre de recherche
F09	Message de contact	Visiteur	Envoyer le formulaire `/contact`	Bloc "Message envoyé !" (pas de toast) ; message visible dans Admin → Messages contact (non lu)
F10	Lecture d'un message	Admin connecté	Marquer lu, puis supprimer	Compteur de non lus mis à jour ; message supprimé
F08	Supprimer un utilisateur	Admin connecté	Supprimer compte	Compte supprimé définitivement

---

BLOC H — Profil kooker et navigation

#	Scénario	Actions	Résultat attendu
H01	Zone d'intervention	Profil kooker : ville = Lyon (suggestion), zone = 20 km, enregistrer	Une adresse de prestation à 17 km trouve le kooker, une adresse à 25 km non
H05	Adresse conservée	Kooker : modifier sa bio et enregistrer	L'adresse du kooker n'est pas effacée
H06	Devenir kooker — adresse	Remplir adresse + ville, valider	Adresse enregistrée sur le profil kooker
H02	Devenir kooker — spécialités	Formulaire devenir kooker	Boutons issus de la liste admin, aucune saisie libre
H03	Header	Ouvrir le site	Pas de lien « Tarification »
H04	Footer	Bas de page	Rubriques Découvrir / Aide & Confiance / Weekook, liens légaux, Instagram et TikTok

---

BLOC G — Messagerie

#	Scénario	Actions	Résultat attendu
G01	Envoyer un message	Connecté, serviceId valide	Envoyer message à kooker
G02	Message à soi-même	—	receiverId = propre userId
G03	Marquer comme lu	Ouvrir conversation	—
G04	Compteur non lus	Message reçu non lu	Voir liste conversations

---

Document généré le 2026-10-02, mis à jour le 2026-10-28 — à maintenir à jour lors de chaque évolution fonctionnelle.